COMP5339 Data Engineering

Assignment 2

Submitted To: Uwe Roehm

Date: 8th November 2025

Submitted By:

Dhruv Lawaniya

Sanskar Agarwal

Victor Kibisu Alusa

1. Introduction

This report presents the implementation and analysis of a data engineering pipeline for the Australian electricity sector as part of the COMP5339 Data Engineering Assignment 2. The project focuses on the end-to-end process of retrieving, cleaning, transforming, integrating, and visualizing electricity generation and emissions data from the Open Electricity platform. The system was designed to acquire per-facility and per-market data from the NEM (National Electricity Market) network for one week in October 2025, covering multiple regions including New South Wales, Queensland, Victoria, South Australia, and Tasmania. Using the Open Electricity REST API, power generation, CO_2 emissions, market price, and demand data were collected and consolidated into a single, structured dataset. Following data acquisition, extensive preprocessing was performed to ensure data consistency and reliability. The cleaned and integrated dataset was then used to simulate a real-time data stream by publishing power and emissions information through an MQTT broker. Finally, a map-based dashboard was developed to visualize the streaming data, dynamically displaying facility locations and their current generation and emission levels.

2. System Description

2.1. Data Ingestion

The initial step of this phase involved declaring key variables such as Interval, Start_Date, End_Date, and Request_Delay, as specified in the instructions. Defining these common variables at the beginning allows for greater flexibility and control throughout the process, enabling easy adjustments when necessary. The data ingestion phase began with retrieving facility data for the “NEM” network. This dataset contains essential information including the facility descriptions, location names, and corresponding facility codes, among other attributes. Subsequently, pre-defined parameters were established by examining the format of individual API calls. These parameters included the Interval, Start_Date, and End_Date (set for the first week of October), as well as relevant metrics such as power, emissions, price, and demand, in addition to the network region. These parameters served as inputs for API calls used to retrieve power, emission, and market data. This stage concluded with printing the headers of all retrieved dataframes which were facilities_df, power_df_raw, emissions_df_raw, and market_raw. This enabled us to verify successful data ingestion and ensure the datasets were correctly structured for subsequent processing.

2.2. Data Processing and Integration

This stage commenced with the grouping of the power and emissions dataframes, followed by a comprehensive cleaning process to ensure data consistency and accuracy. The thorough cleaning of the combined dataframe was crucial in preventing discrepancies during subsequent analytical steps. The facilities_df was then refined by removing unnecessary columns and extracting geolocation data. Additional attributes, such as fuel type, were also extracted to facilitate more meaningful visualizations in later stages. After cleaning, the facilities dataframe was merged with the power_emission dataframe, resulting in an integrated dataset that captured both facility-specific and performance-related information. Similar cleaning and preprocessing steps were performed on the price and demand datasets. Once processed, these were also merged with the previously combined dataframe, creating a unified dataset encompassing all relevant variables. To streamline data management, only a single CSV file saved as *power_emission.csv*, was retained. This decision minimized the number of files required, simplified the execution of the code by any user, and ensured that all necessary data was successfully consolidated into one cohesive and accessible dataset.

2.3. Data Publishing/Subscription

Data from ‘power_emissison.csv’ is then streamed over an MQTT Broker as a combined message containing all necessary information required for the next task (as specified in the spec). MQTT uses QoS 0 which means that messages are neither stored nor acknowledged, the ‘at most once’ guarantee means there is no persistence, i.e. there is no real benefit to splitting messaged as static attributes (name, coordinate, or fuel type) or dynamic attributes (power, emission) into separate channels due to a 0.1s delay imposed between each publish, therefore each message has the full facility record. Before publishing, a challenge we faced was parsing units field returned by OpenElectricity API, the units identified are embedded into a string representation of a list of dictionaries, these were transformed to a python list of dictionaries for unit codes so that the subscriber can display unit level data when a facility is selected. The subscriber then subscribed to the published topic and decodes received messages from JSON into native Python types, ensuring values are correctly interpreted (e.g., power as float, timestamp as datetime). The message is then separated into static and dynamic components, static attributes are stored onto a shared facility dictionary which checks if an inputted facility is being ingested for the first time, upon ingested we integrate facilities latitude and longitude with *A1_Dataset.csv* and keep its published values as a fallback, dynamic values are updated continuously as they arrive. This allows the fields that change to be re-rendered on the map individually by a scaling factor to their total power/emission value.

2.4. Data Visualization

The dashboard presents an interactive map view of facilities across NEM, updated continuously as new data is received by the subscriber. The facilities are displayed using their latitude and longitude integrated from A1 or the fallback option of MQTT messages and stored in the facilities dictionary. The interface for this is Python's Plotly, which provides browser-based

interactivity without requiring a full web application. The marker size of each facility is factored by total power or total emissions value. The user can toggle between power or emissions toggle where the hover will display facility name and latest values, and a click will produce a static snapshot providing detailed information about static fuel technology, facility unit codes, and dynamic current power, current emission, total power, total emission and state level values of power price and power demand on the GUI.

The visual layer reads directly from the in-memory facilities dictionary, which is being updated by the subscription stream, only dynamic attributes refresh on the map. This helps by avoiding a full re-render of the map which produces a flickering effect and instead gives a smoother evolving state of power and emission generation in real time by re-rendering individual facility markers by their total power or emission.

2.5. Continuous Execution and Replay

The publisher is publishing a 0.1 second delay between each published message to preserve event-time ordering and avoid overloading the server. Once all messages from *power_emission.csv* are published, the system waits 60 seconds and then replays from the beginning of the dataset to emulate real-time, unbounded data stream via a continuous replay loop. Duplicate facilities were already removed during data cleaning via *df.drop_duplicates()*, we further ensured only updated values are pushed by checking if both power and emission are non-zero. A check is also placed via a *last_seen* lookup dictionary that checks most recent published power and emission values for each facility during the stream run, a new message is published only if at least one of these values has changed up to a small float tolerance to prevent redundant publishing while preserving real time behavior. On replay *last_seen* resets, which is expected, since the replay represents a new live stream (therefore a 60s pause) rather than a continuation of the previous one. Our prior cleaning and further checks ensured that the MQTT servers' latency is not compromised, and the dashboard receives continuous, smooth streams of data, sustaining a real time visualization pipeline.

3. Data Acquisition, Cleaning, and Transformation

3.1. Data Retrieval

To retrieve the power and emissions data, it was necessary to provide the facility_code parameter. To obtain these facility codes, the list of all facilities within the NEM network was first retrieved using the fetch_all_facilities function (referenced as section 1.2 in the code file). This function returned key parameters for each facility, including name, network_id, network_region, description, location, code, units, created_at, and updated_at. In accordance with the assignment instructions, several constants were defined for the data retrieval process: Interval was set to 5 minutes, Start_Date to 1 October 2025, End_Date to 8 October 2025 (not inclusive), and Request_Delay to 60 seconds.

To extract power generation and emissions data for each facility, two dedicated functions (sections 1.2.1 and 1.2.2 in the code file) were implemented. These functions utilized the parameters to systematically fetch data for all available facilities. In total, data retrieval was attempted for 580 facilities across the NEM network. However, several facilities did not return data within the specified date range, as many power plants operate on a seasonal basis. Robust error-handling mechanisms were incorporated to ensure the functions continued execution even when data for certain facilities was unavailable, thereby preventing process interruptions. For retrieving per-market power price and demand data, the network_region parameter was used instead of facility_code. This approach produced highly structured datasets with exactly 2,016 rows per parameter per day. It was also observed that this market-level data was significantly cleaner and more consistent compared to the power and emissions datasets.

3.2. Data Cleaning, Transformation, and Integration

The objective of this stage was to consolidate all datasets into a single CSV file, facilitating efficient and streamlined data publishing through MQTT. This is displayed in *Table 1* and further discussed in detail below.

DataFrame	Description	Rows	Columns
facilities_df	Facility metadata from the NEM network including name, region, and location.	580	9
power_df_raw	Raw per-facility power generation data at 5-minute intervals.	979365	4
emissions_df_raw	Raw per-facility emissions data for the same timestamps.	977349	4
market_raw	Raw per-facility emissions data for the same timestamps.	70560	7
power_emission_raw_stage_1	Initial merged dataset of power and emissions using outer join.	1121429	4

power_emission_raw_stage_2	Cleaned version after removing invalid and redundant rows.	691519	4
facilities_df_stage_1	Cleaned facilities data with extracted coordinates and fueltech_id.	580	8
power_emission_raw_stage_3	Combined facilities and power–emission data on facility_id.	691519	12
power_emission_raw_stage_4	Further cleaned dataset with standardized timestamps.	691519	10
power_emission_final	Final version after duplicate removal on facility_id and timestamp.	406845	10
merged_df	Fully integrated dataset of facility, market, power, and emissions data.	406845	12

Table 1: Data Cleaning and Transformation Process

3.2.1. Cleaning and Merging Power and Emissions Data

This step began by combining the power and emissions dataframes using an outer join. The outer join ensured that all rows were retained, even when either power or emission values were missing for a given timestamp. For data cleaning, the absolute values of *power* were taken since negative readings were observed, which are not physically meaningful. Removing such data was avoided, as the purpose of this task was to simulate a continuous data stream; hence, taking absolute values was a logical choice.

Timestamps where both power and emission values were zero were removed, as including them would not contribute meaningful insights in visualization. The *units* column, which contained measurement units such as "mW" or "t", was also dropped, as it did not provide analytical value. It is worth noting that this *units* column was distinct from the *units* column present in *facilities_df*. Further cleaning involved rounding the *power* and *emission* values to three decimal places and formatting them consistently. Duplicate rows with identical *timestamp*, *power*, and *emission* values were removed. By the end of this stage, a smaller and significantly cleaner dataframe was obtained.

3.2.2. Refining Facility Data

The facility data retrieved from the API contained extensive details, many of which were unnecessary for the scope of this assignment. Cleaning *facilities_df* began by dropping irrelevant columns such as *description*, *created_at*, and *updated_at*. The *location* field was then split into *latitude* and *longitude*, which were retained for later use in map-based visualizations. The *network_region* values had a trailing "1" that was removed for consistency. Additionally, the *units* column contained a list of small generation units within each power plant. From this, the *fueltech_id* of the first listed unit was extracted as the facility's primary fuel source. Given that it is uncommon for a single power plant to employ multiple energy sources (e.g., coal and solar simultaneously), extracting the first entry was deemed a reasonable simplification.

3.2.3. Integrating Facilities with Power and Emissions Data

The cleaned *facilities_df* was then left joined with the *power_emission* dataframe using the *facility_id* as the key. Duplicate records were dropped based on *timestamp* and *facility_id*, retaining only the first occurrence. This ensured a one-to-one correspondence between each facility and timestamp, simplifying subsequent visualization steps. This method was found to be both efficient and effective for achieving a clean and consolidated dataset.

3.2.4. Cleaning Market Data (Price and Demand)

Like the earlier cleaning procedures, the *market_raw* dataframe underwent several refinements. Trailing "1" characters were removed from *network_region* codes, duplicate timestamps were deleted, and the *network* column was dropped since all data originated from the NEM network. Certain columns were renamed for clarity, the *units* column was removed, and the data was sorted by *timestamp* and *network_region* to maintain consistency.

3.2.5. Final Integration of Market Data

The preprocessing stage concluded with the integration of the cleaned market data into the existing dataset. A left join was performed on *timestamp* and *network_region* (the latter obtained from *facilities_df*), resulting in a fully consolidated, cleaned, and integrated dataframe which was sorted by timestamp and network region, ready for subsequent analysis and publication, which was saved as the final "power_emission.csv".

4. Data Integration

Our facilities data from Assignment 1 was read as a csv and filtered to retain only NEM Facilities with valid latitudes and longitudes. Facility names were then stored in a dictionary with name normalization to match the style of naming done on messages received by the MQTT subscriber. This lookup dictionary maps every facility name to its coordinates and was matched

against each newly ingested facilities name, to render facilities coordinates as a static variable with the geolocation given by the MQTT message as a fallback option.

5. Data Visualisation

5.1. Visualisation Overview

Detailed screenshots of the visualisation are made available in the appendix, the visualisation interface provides a real-time map of power generation and emissions across NEM. *Figure 1* shows the start-stop buttons that can be used to subscribe and unsubscribe to the MQTT data stream and the toggle which helps us switch between power and emission views. Facilities are represented as dynamic bubble sizes by the total power or emission of the facility coloured by fuel type. *Figure 2* zooms in on a facility Mt. Piper and shows a hover view of the current power and emissions readings, on clicking on the facility a detailed popup appears on the GUI where static attributes like unit code, and facility name are highlighted in blue, and dynamic attributes like current price, total power, and current power are highlighted in red. *Figure 3* then shows the same facility after two more time-stamp values have been ingested to display the updated values on the hover and the detailed pop-up. *Figure 4* shows a real-time render of the dashboard on the emissions panel.

5.2. Design Principles

5.2.1. Homogeneity of Units

We have assumed that all units within a facility are of the same fuel technology group, it is unlikely that a solar facility with contain a coal plant along with two solar panels. This assumption has been empirically validated in *INGEST_CLEAN_PUBLISH.ipynb* which shows 284 of the 304(93%) unique facilities fetched share uniform fueltech classification.

5.2.2. Market Data Integration

Live market price and demand are state-level attributes updating independently of facility power and emissions. Due to a single topic + message constrained imposed in the spec, the arrival order is asynchronous with these arriving either right before or right after the live facility updates depending on sorting. Further facilities appear live with latest values even after they stop generating (e.g. a facility stops generation on day two will still stay on the map as a live value on day 5 while others facilities and market values are on day 5), the display would create mismatches and render false information. Therefore, these values are displayed on-demand on-click panel to ensure live indicators are not misleading.

5.2.3. Continuous Loop for Unbounded Stream

The replay is configured to simulate an unbounded data stream, once it runs out of data, it takes a 60 second pause and then replays data from the beginning of *power_emisssion.csv*, to simulate new data coming. The *last_seen* state resets, which is intentional, as the cycle is representing a new real time stream. This setup was tested robustly on multiple subsets of the dataset to ensure stable, unbounded streaming behaviour.

6. Conclusion

This project successfully demonstrates an end-to-end data engineering pipeline to display real-time electricity generation and emission monitoring across NEM. The system design and design decisions made ensure performance stability, clarity of display, and ease of adaption for the future. Future extensions of the project could include a multi-topic based streaming to separate static and dynamic fields to reduce message size and improve scalability for larger messages and datasets, and addition of a regional or temporal analytical layer to add historic trend analysis and forecasting to extend our dashboard beyond real-time monitoring to a decision making and support planning insights platform.

APPENDIX

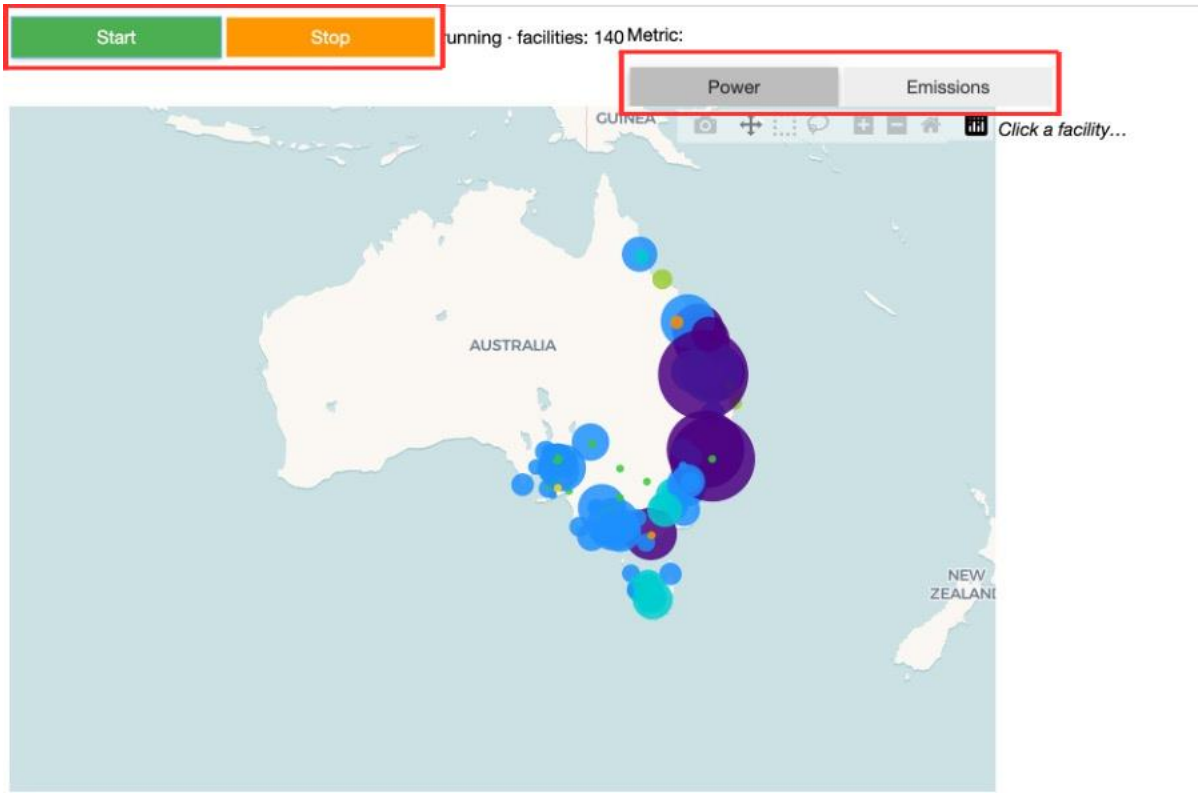

Figure 1: Live dashboard interface with Start/Stop stream controls and Power/Emissions toggle

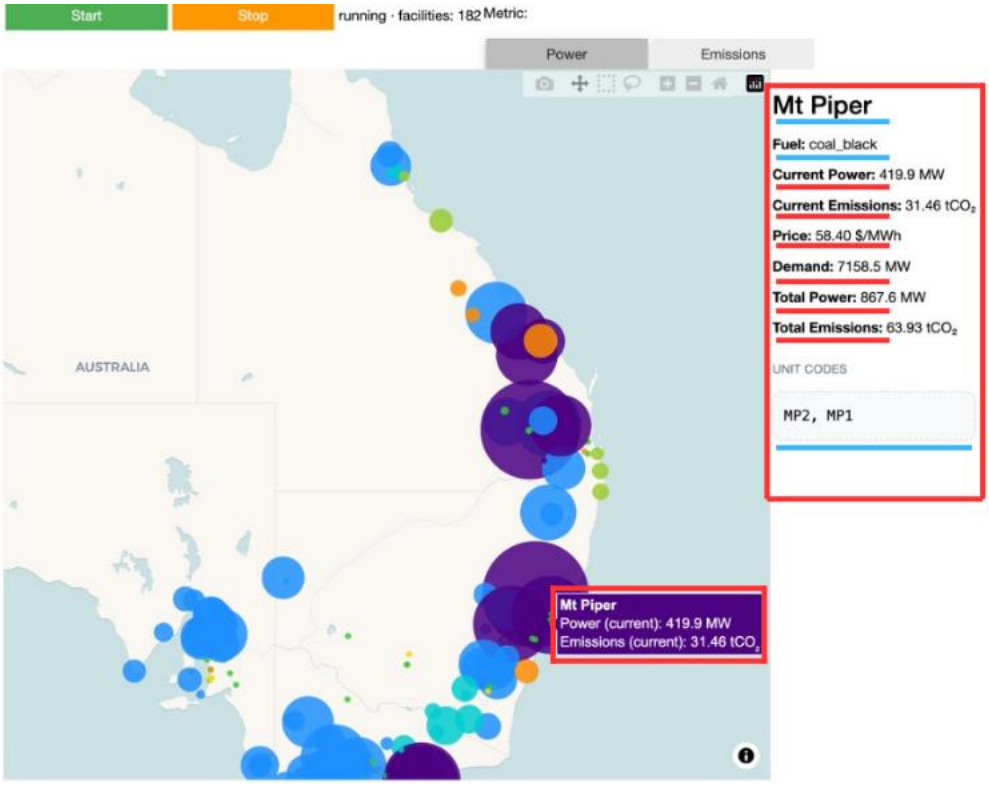

Figure 2: Emissions view showing facility-level tooltips on hover

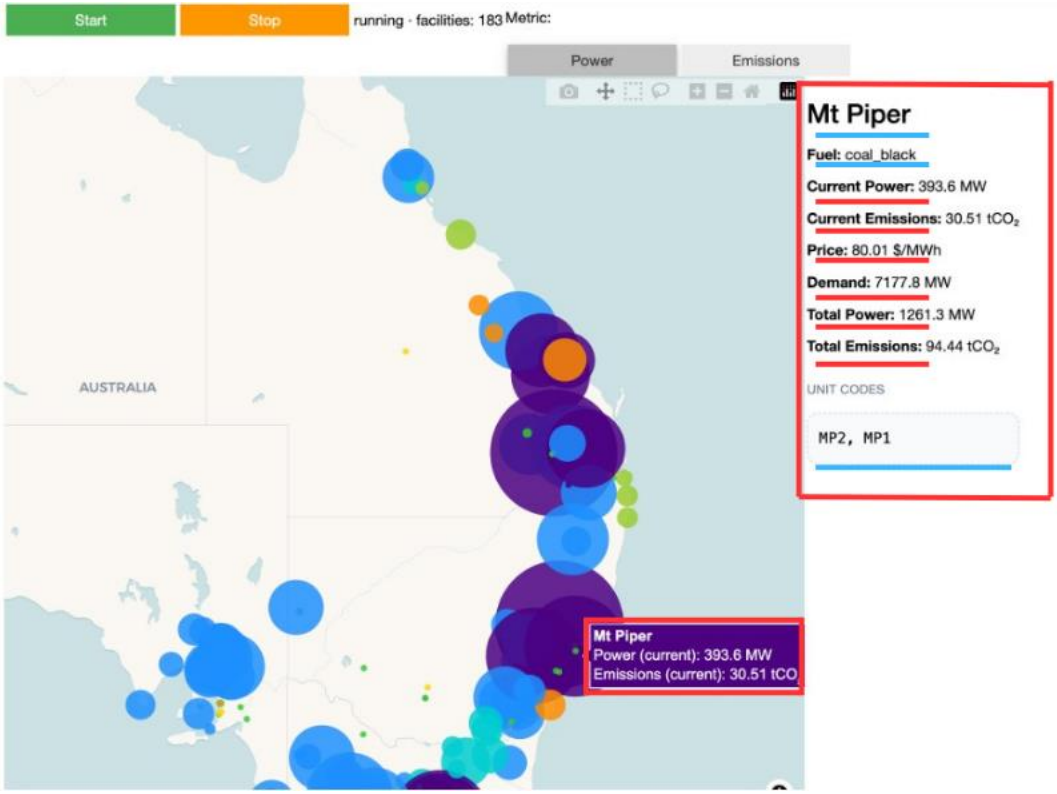

Figure 3: updated detail panel displaying operational statistics and unit information.

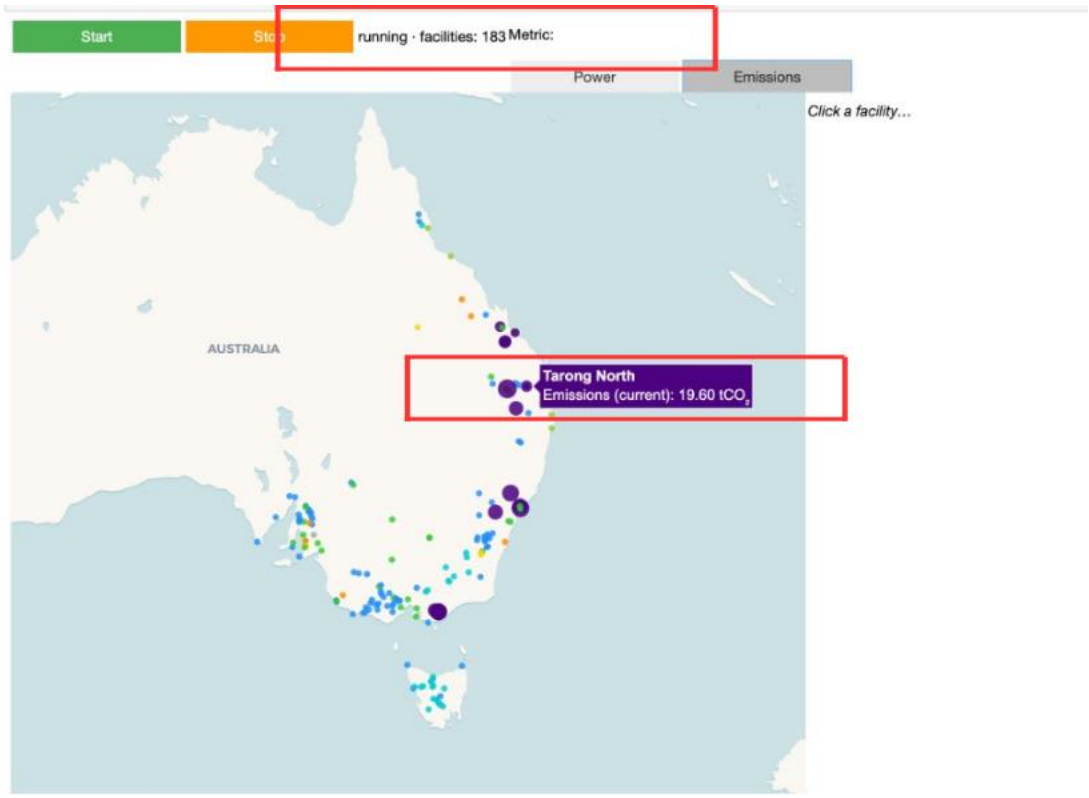

Figure 4: Emission Toggle, Real-time updates to map and facility metrics as new data is streamed.

Contribution Summary

Name	Tasks	Contribution
Dhruv	Data Retrieval	Major Contribution
	Data Cleaning	
	Data Transformation	
	Integration	
	Wrote Report	
Sanskar	Subscription	Major Contribution
	Decode	
	Visualisation	
	Continuous Execution + Units for Publish	
	Wrote Report	
Victor	Data publishing	Minor Contribution

Declaration of AI

We, Group 10(Tutorial 15), hereby declare we did not use GenAI to write our report. Ideas presented are our own, we have used ChatGPT o5 (https://chatgpt.com) and Google Gemini (gemini.google.com) for content and idea refinement as well as running our report against the rubric to see what ideas should be further elaborated.